MANAGEMENT BRIEFING | 13 OCT 2026

ROCKE-3D to JAX: progress, proof, decisions

Goal: a JAX implementation of the P2SAoM40 model that is demonstrably faithful and faster than the original Fortran.

2.13-2.21 s

Tagged release: JAX / step

Fortran: 2.24-2.28 s. Two exclusive V100 nodes: near parity, not a decisive speed win.

0.524 s

Optimized branch: JAX / step

Fortran: 2.285 s. One matched run, steps 2-107: 4.4x steady-state speedup.

933 s

Fast-run cold step 0

Plus 96 s at step 1. Estimated payback vs Fortran: about 580 steps / 12 days.

Bottom line: a credible speed result exists, but only on an unmerged, specially flagged configuration. Short runs lose to compilation; the fast path still needs validation before release.

Keep these three labels distinct

RELEASE = native-radiation JAX, near steady-state parity. FAST CANDIDATE = 4.4x steady-state, unmerged. QUALIFICATION = fast-path tests, repeat benchmark, fidelity and longer runs still open.

Sources: RELEASE_NATIVE_RADIATION_20261012; SPEED_STEP_TIME_LOG_20261012 s13-14; SPEED_NEXT_STEPS_20261013

How the project got here

A deliberate progression: establish a real Fortran reference, couple the physics, then attack the measured bottleneck.

1 TRUST THE REFERENCE

Rebuilt and replayed real Fortran runs; ported components against recorded calls.

>

2 COUPLE THE MODEL

Assembled a multi-day JAX-driven hybrid; Fortran RADIA/SOCRATES remained live.

>

3 PORT RADIATION

JAX SOCRATES/RADIA matched same-packet Fortran outputs at rounding level; coupled native run completed.

>

4 OPTIMIZE

Measured and removed host/dispatch bottlenecks; one configuration reached 0.524 s/step.

Progress is real, but maturity differs: the released native-radiation build is near parity; the larger speedup belongs to a separate experimental branch.

Sources: PROJECT_CHRONOLOGY; RELEASE_NATIVE_RADIATION_20261012; SOCRATES_R2_DUMPS s12; SPEED_STEP_TIME_LOG_20261012

What the evidence says—and does not say

The speed number is not the only gate: fidelity, reproducibility and run length determine whether it is useful.

SUPPORTED BY MEASUREMENTS

- Native JAX radiation is wired into the coupled run; it reduced median radiation-step wall time from 10.0 s to 1.9 s.
- Optimized matched benchmark: same exclusive V100 node, Fortran x2 then JAX, same 108-step window; 2.285 vs 0.524 s/step after startup.
- Two-day fast run completed; its score pattern is close to baseline and finite at each step.
- Release native configuration independently measured on two V100 nodes: near parity with Fortran.

NOT YET ESTABLISHED

- Strict 2x fidelity gate remains NOT MET: early QCI outlier ratio 2.47 at step 3 (baseline 2.37).
- Fast-flag configuration has not run the full regression suite; suite evidence is for flags off.
- No repeat of the exact matched fast benchmark; 0.434 s is a separate profile-wrapper run, not a second matched run.
- No long free run on the fast flags. The 30-day shadow run used the non-fast configuration and had invalid model states after step 1288.

Scope: nov26 start only; five recorded step-0 inputs; V100 / float64. The fast branch uses an experimental Numba worker environment. Do not generalize to other seasons, GPUs or production runs yet.

Sources: SPEED_STEP_TIME_LOG_20261012 s13-14; ACCEPTANCE_CRITERIA s9-10; SOCRATES_TRIAL_RUN_RECIPE s10; SPEED_RUN_RECIPE_20261013

Management input needed to proceed usefully

The next work should answer the decisions that determine whether this speedup matters and what “good enough” means.

1 DEFINE THE TARGET USE

+ What is a representative production run length and restart cadence?
+ Is a ~12-day estimated payback useful for the intended workload, or is cold-start latency a blocker?
+ Is V100 the deployment target, or should qualification include another device?

2 SET THE SCIENCE BAR

+ Keep the original strict 2x rule for release, or explicitly approve a separate experimental criterion?
+ Who owns the scientific disposition of the QCI step-3 outlier and the branch’s 2.37 -> 2.47 shift?
+ Which additional dates / run duration are necessary for a useful go/no-go?

3 AUTHORIZE A BOUNDED QUALIFICATION

+ Reserve exclusive V100 runs for repeat benchmark and fast-path regression.
+ Decide whether to support the Numba worker dependency in a maintained environment.

4 CONTROL THE RELEASE DECISION

+ Keep the tagged near-parity release as the stable baseline.
+ Merge the fast branch only after agreed test, repeatability, fidelity and long-run gates pass.

Recommended posture: fund a time-boxed qualification, not a production claim. Preserve the release; promote the fast path only against explicit gates.

Sources: Owner input requested; SPEED_NEXT_STEPS_20261013; RELEASE_NATIVE_RADIATION_20261012

Proposed gates: turn a promising result into a decision

Small, ordered checks; stop or re-scope if the evidence does not support deployment.

A	**REPRODUCE**	Repeat the locked matched benchmark on exclusive V100 nodes; publish run-to-run spread and amortized time for the agreed workload.
B	**QUALIFY THE CONFIG**	Run regression with the exact fast flags; resolve/record the known order-dependent test; package the Numba worker dependency.
C	**CHECK FIDELITY**	Score the fast path against the accepted rule on agreed starts; preserve strict and any separately authorized experimental status.
D	**TEST LONGER RUNS**	Run the fast configuration beyond two days; investigate `qdynam_do_z_extra` before claiming the 30-day trajectory is valid.
E	**PROMOTE OR HOLD**	Compare evidence to the target workload and science bar; only then decide merge, release and operational support.

No schedule is implied here: the owner should set the run count, dates, and acceptance limits after choosing the target workload.

Sources: SPEED_NEXT_STEPS_20261013; SPEED_STEP_TIME_LOG_20261012 s13; SOCRATES_TRIAL_RUN_RECIPE s10